

Skytree PMML Scorer

Release 15.4.0

SKYTREE®

Notices

Copyright

Copyright 2014-2016 Skytree, Inc. All rights reserved.

Information in this document is furnished only under a Customer License Agreement or non-disclosure agreement and may be used or copied only in accordance with the terms of either such agreement. The software described in this document is owned by Skytree, Inc. and protected by U.S. copyright law and may not be copied on any medium except as specifically authorized in the Customer License Agreement or non-disclosure agreement.

Disclaimer

Skytree, Inc. reserves the right to revise this document and to make changes from time to time without any obligation of any kind to notify any person of such revisions or changes.

Trademarks

SKYTREE®, the Skytree logo, THE MACHINE LEARNING COMPANY®, THE DATA DRIVEN ENTERPRISE™, Skytree Infinity™, AutoModel™, and AutoDocumentation™ are trademarks or registered trademarks of Skytree, Inc. ORACLE®, JAVA®, and JDK™ are trademarks or registered trademarks of Oracle and/or its affiliates. HADOOP®, HIVE™, PIG™, HBASE™, SPARK™, TOMCAT™, and ZOOKEEPER™ are trademarks or registered trademarks of the Apache Software Foundation. MAPR® is a registered trademark of MapR Technologies, Inc. Python™ is a trademark of the Python Software Foundation. Third-party product names and related marks used in this document are trademarks or registered trademarks of their respective owners. Use of third-party product names and marks does not imply any affiliation or endorsement by such parties.

Company Information:

Skytree, Inc.
1731 Technology Drive, Suite 700
San Jose, CA 95110
408.392.9300
www.skytree.net

Customers with a support contact can contact support using any of the following methods:

- Log in to support.skytree.net.
- Send questions to support@skytree.net.
- Call 408.392.9300 and select option 3.
- You can also view additional Skytree documentation by logging on to support.skytree.net.

Freemium customers can send inquiries to the LinkedIn® Community Group for Skytree:

<https://www.linkedin.com/groups/4468542>

Revision Date: February 28, 2016

Contents

About this Document 1

Software Requirements 1

Running the PMML in Batch Mode 1

Classification Model Example 1

Regression Model Example 2

Integrating the PMML Scorer into Applications 3

 JPMML 3

 Guided Examples 3

 Threading 5

 Other Guidelines 5

Skytree PMML Evaluator Performance 5

About this Document

This document describes how to get started using the Skytree PMML Scorer. Using the included files in the “examples” folder, this document will take you through the following steps:

1. Running the PMML scorer in standalone batch model for a classification problem and explaining the outputs.
2. Running the PMML scorer in standalone batch model for a regression problem and explaining the outputs.
3. How to integrate the PMML scorer into an application by linking the JAR
 - a. For point by point prediction suited for real time applications
 - b. For batch prediction within an application

Software Requirements

Java - One of:

- Open JDK™ (Java Development Kit) 1.7 or higher
- Oracle Java 7 or higher

Running the PMML in Batch Mode

The PMML package contains the following JAR files:

- skytree-evaluator.jar

This JAR contains all the execution logic for PMML machine learning models. It also contains a driver program in the class `com.skytree.tools.evaluator.CliEvaluator`, which can be run using the command line.

- skytree.jar

This JAR contains the following two utilities, which are utilized in the CLI driver program mentioned above.

- Command Line Options Parsing Logic
- A String Tokenizer to tokenize the points in the input test data file.

To run the driver program, the following command can be used:

```
java -cp skytree-evaluator.jar:skytree.jar com.skytree.tools.evaluator.CliEvaluator
```

This will display the help menu.

Classification Model Example

To make classification predictions on a test dataset from a model built on a corresponding training dataset, we will utilize the following files:

- **income.data:** This file contains the raw training data points, which was used to train the model. For the classification example, the column to be predicted is the “yearly-income” column, which contains 2 values { “<=50K” , “>50K”}. Thus this is a binary classification problem.
- **income.test:** This file contains the points on which we want to make predictions.
- **income.data.classification.pmml:** This file contains a Gradient Boosted Tree model output by Skytree to predict the “yearly-income” column.

Run the following command:

```
java -cp skytree-evaluator.jar:skytree.jar com.skytree.tools.evaluator.CliEvaluator \
-data_in income.test \
-model_in income.data.classification.pmml.xml \
-results_out classifications.csv \
-delimiter COMMA
```

This will create a file named “classifications.csv”, which will contain predictions for each row of the input dataset “income.test”. This file contains 3 columns:

- **treeValues:** These are the raw values of the scores returned from the Gradient Boosted Tree Model.
- **probability_1:** This is the predicted probability of being class 1 as predicted by the model,
- **probability_0:** This is the predicted probability of not being of class 1.

Regression Model Example

To make regression predictions on a test dataset from a model built on a corresponding training dataset, we will utilize the following files:

- **income.data:** This file contains the raw training data points which were used to train the model.. For the regression example, the column to be predicted is the “age”.
- **income.test:** This file contains the points on which we want to make predictions.
- **income.data.regression.pmml:** This file contains a Gradient Boosted Tree Regression model output by Skytree to predict the “age” column.

Run the following command:

```
java -cp skytree-evaluator.jar:skytree.jar com.skytree.tools.evaluator.CliEvaluator \
-data_in income.test \
-model_in income.data.regression.pmml.xml \
-results_out regression.csv \
-delimiter COMMA
```

This will create a file name “regression.csv”, which will contain predictions for each row of the input dataset “income.test”. This file contains a single column:

- **target:** This is the predicted target value, in this case the “age” as predicted by the model.

Integrating the PMML Scorer into Applications

The Skytree PMML Scorer software can be imported into the build process of your Java application. For a single model the following process is necessary:

1. Setting up the PMML evaluator. The following inputs are needed.
 - The location of the PMML file.
 - The column names for the test data as input to the evaluator.
2. Other setup parameters will depend on the application logic and include:
 - The missing value identifier that the input data uses.
 - The delimiter used if any in the input test data.
 - Case sensitivity of input data.
3. Once the PMML evaluator is setup, it has logic to predict one test point at a time.

JPMML

Skytree PMML Evaluator makes use of the JPMML Evaluator project. Please visit <https://github.com/jpmml/jpmml-evaluator> for more information on the JPMML Evaluator project. Any model produced by Skytree software will work with the Skytree PMML Evaluator, which wraps parts of the JPMML package. All models produced by Skytree are tested against the version of JPMML distributed with our software and meet our standards for reproducing results exactly as produced directly by the Skytree scoring engine CLI/SDK/UI.

Guided Examples

The following code snippets illustrate each step in loading, predicting and producing outputs from a model in PMML format using the PMML Evaluator software. These walk through all the steps needed to load and setup a model and then make predictions in an iterative manner. It is very similar to the code that is used to run the software in batch mode in the CLI and the same logic is easily replicated to predict for a single test point in a real time scoring environment.

Imports

```
import java.io.FileInputStream;
import java.io.InputStream;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import javax.xml.transform.Source;
import org.xml.sax.InputSource;
import org.dmg.pmml.FieldName;
import org.dmg.pmml.PMML;
import org.jpmml.evaluator.Evaluator;
import org.jpmml.evaluator.EvaluatorUtil;
import org.jpmml.evaluator.FieldValue;
import org.jpmml.evaluator.ModelEvaluatorFactory;
import org.jpmml.manager.PMMLManager;
import org.jpmml.model.ImportFilter;
```

```
import org.jpmmml.model.JAXBUtil;
```

Loading a Model

```
// Obtain the name of of the PMML model file to load String inputModel = ...;

// The Evaluator object performs all model evaluation tasks Evaluator evaluator;

{
    // The PMML object stores the XML content of the model
    PMML pmm1;

    // Read the XML content from file
    InputStream is = new FileInputStream(inputModel);
    try {
        Source source = ImportFilter.apply(new InputSource(is));
        pmm1 = JAXBUtil.unmarshalPMML(source);
    } finally {
        if (is != null) is.close();
    }

    // Create an Evaluator object based on the PMML model
    PMMLManager pmm1Manager = new PMMLManager(pmm1);
    evaluator = (Evaluator)pmm1Manager.getModelManager(
        ModelEvaluatorFactory.getInstance());
}
```

Evaluating a Model

```
// Extract a list of required input FieldNames from the Evaluator object
//
// In our example, we may want to rearrange these FieldNames in order to
// match the order of the input columns. orderedInputFields may be larger
// than inputFields, with null entries indicating skipped columns.
List<FieldName> inputFields = evaluator.getActiveFields();
FieldName[] orderedInputFields = ...;

// Extract a list of all output FieldNames from the Evaluator object
List<FieldName> outputFields = new ArrayList<FieldName>();
outputFields.addAll(evaluator.getTargetFields());
outputFields.addAll(evaluator.getOutputFields());

while (...) {
    // Obtain values pertaining to one row of input as in our example
    //
    // Input values need not in general be formatted as strings;
    // EvaluatorUtil.prepare (see below) can accept generic Objects as
    // appropriate for the input field defined in the PMML model.
    String[] orderedInputValues = ...;

    // Construct a map from FieldNames to their corresponding values
    Map<FieldName, FieldValue> fieldvaluesMap =
        new LinkedHashMap<FieldName, FieldValue>();
    for (int i = 0; i < orderedInputFields.length; ++i) {
        // Respect input column order, skipping unused columns
        FieldName inputField = orderedInputFields[i];
        if (name == null) continue;
```



```

    // Prepare and insert a key-value pair pertaining to the field
    FieldValue value = EvaluatorUtil.prepare(
        evaluator, inputField, orderedInputValues[i]);
    fieldValuesMap.put(inputField, value);
}

// Evaluate the model on the prepared input
Map<FieldName, ?> resultsMap = evaluator.evaluate(fieldValuesMap);

// Extract and process the output values
//
// In general, only specific output fields may be of interest; it is by
// no means necessary to visit and decode all results. Alternately, it is
// possible to call EvaluatorUtil.decode directly on resultsMap,
// obtaining
// a Map<String, ?> containing all output field name-value pairs.
//
// Also, like inputs, output values are not necessarily stored as Strings
// if a given output field is known to have Double type, its value may be
// obtained by direct cast to Double.
for (FieldName outputField : outputFields) {
    String outputValue = String.valueOf(
        EvaluatorUtil.decode(resultsMap.get(outputField)));
    ...
}
}

```

Threading

JPMML does not use threading internally, but it is thread safe. Applications can call `Evaluator.evaluate` concurrently with same Evaluator object from different threads in order to compute predictions for multiple input rows in parallel. In this case, it is only necessary to load the PMML model once to construct the shared Evaluator object, conserving memory while still enabling parallelized evaluation.

Alternately, you can launch multiple, separate Java processes to compute predictions in parallel or even in the distributed setting. In this case, it is necessary to distribute input rows across the Java processes using an external, load-balancing task dispatcher. Also, each Java process must load its own copy of the PMML model in order to construct a unique Evaluator object. This results in increased memory requirements proportional to the number of processes, but has the benefit that those processes may run on separate servers with access to separate resource pools. Additionally, with a sufficiently capable external task dispatcher, a failure on one process or server does not necessarily result in the entire prediction system going down.

Other Guidelines

- The evaluate function throws an Exception if the test data point is malformed. Surrounding code should catch these and handle them appropriately.
- For further reading and documentation on the API, see <https://github.com/jpmml/jpmml-evaluator>.

Skytree PMML Evaluator Performance

Using the adult/income dataset on an internal environment (128GB RAM) with a single PMML Evaluator thread:

- GBT autotuned (934 trees, max 13 nodes per tree):

latency 2.25ms, throughput 500/s per thread

- SVM autotuned (on 105 horizontalized columns):

latency 0.875ms, throughput 1900/s per thread

The latency is measured as the average round-trip time (recv/read a line, evaluate, and send/write a result) - the bandwidth is measured as the number of evaluations that could be performed in batch.

When our PMML Evaluator library is integrated into the customer application, the customer has total freedom to use multiple threads (dynamically created or using a thread pool) to concurrently invoke the evaluator API of different points or batches of points – this will give them throughput multiplication as the number of threads are increased (within limits based on number of cores and memory on the node).

Users can run their application on a cluster of nodes with one application process per node, where they can further multiply the evaluation throughput since each process that links this library will have loaded the same model and can support evaluation via multiple threads within that process.